Solving Enhanced Frozen Lake via Memory-Based and Distributional Reinforcement Learning

Shengkai (Kevin) Tao, Xiyuan (Sophia) Shen
School of Data Science And Society
UNC Chapel Hill

Abstract

While standard reinforcement learning (RL) models excel in static environments, real-world applications demand resilience against uncertainty. From search-and-rescue drones to autonomous underwater vehicles, agents must navigate partial observability and dynamic environmental shifts. This project addresses these challenges using an Enhanced Frozen Lake simulation, which transforms the standard environment into a Partially Observable Markov Decision Process (POMDP) with non-stationary dynamics. We integrate memory-based architectures with curiosity-driven exploration and distributional RL to bridge the gap between simple grid-world navigation and complex, risk-aware decision making. Our results demonstrate that the combination of temporal memory and distributional return estimation significantly enhances agent stability and success rates in stochastic environments.

1 Problem Statement & Motivation

Motivation: Modern RL agents often struggle when faced with the "deadly triad" of partial observability, non-stationary dynamics, and sparse, high-penalty rewards. Our project is motivated by the need for agents that do not simply memorize a map, but instead develop an internal representation of environmental risks. By using the Enhanced Frozen Lake simulation, we test whether an agent can remain sufficiently adventurous to find the goal while developing the intelligence required to survive unpredictable challenges.

Frozen Lake: In the standard environment, an agent travels across a 4x4 grid, attempting to reach a goal while avoiding holes. Movement is restricted to cardinal directions (up, down, left, right).

Enhanced Frozen Lake: We extend the standard environment by introducing three primary complications that elevate the task to a POMDP. Each challenge is addressed with a specific architectural solution, summarized in Table 1.

Table 1: Challenges and Proposed Solutions

Challenge	**Description**	**Proposed Solution**
Partial Observability	Agent only sees a 5×5 window; the global state is unknown.	DRQN (LSTM Memory)
Non-Stationary Dynamics	A "wind" variable shifts every 1,000 episodes, altering transition probabilities.	QR-DRQN (Distributional RL)
Costly Exploration	High penalties for failure discourage exploration in new maps.	RND-DRQN (Curiosity)

2 Background and Related Work

2.1 Deep Q-Network (DQN) Baseline

"Human-Level Control through Deep Reinforcement Learning" [4]

Mnih et al. established the modern baseline for deep RL by combining Q-learning with convolutional neural networks. DQN introduced two critical innovations: 1) **Experience Replay**, which stores transitions in a buffer to break temporal correlations through random sampling, and 2) a **Target Network**, which provides stable optimization targets. While DQN excels in fully observable environments, it struggles with POMDPs and stochastic dynamics where single-step observations are insufficient for decision making.

2.2 Deep Recurrent Q-Network (DRQN)

"Deep Recurrent Q-Learning for Partially Observable MDPs" [3]

Hausknecht and Stone addressed the limitations of DQN in partially observable environments by replacing the initial fully connected layer of the neural network with a Long Short-Term Memory (LSTM) recurrent layer. This allows the agent to maintain a hidden state that integrates observations over time, effectively building a temporal memory. In our work, we utilize DRQN as the foundational architecture to handle the 5×5 vision constraint.

2.3 Distributional RL (QR-DQN)

"Implicit Quantile Networks for Distributional Reinforcement Learning" [2]

Standard RL models predict the *expected* value of future rewards. In contrast, Distributional RL predicts the full probability distribution of returns. Quantile Regression DQN (QR-DQN) approximates this distribution using a set of discretely spaced quantiles. This approach is particularly effective in stochastic environments—such as our "windy" Frozen Lake—where it allows the agent to distinguish between mean performance and risk-heavy tails.

2.4 RL in Non-Stationary Environments

"Reinforcement Learning in Non-Stationary Environments" [5]

Handling shifting dynamics is a core challenge in real-world RL. Padakandla et al. proposed methods for detecting environmental change points (e.g., shifts in wind direction) and adapting policies accordingly. Our implementation addresses non-stationarity by providing the agent with explicit wind embeddings and utilizing distributional returns to absorb aleatoric uncertainty.

2.5 Random Network Distillation (RND)

"Exploration by Random Network Distillation" [1]

Exploration in high-penalty environments is often stifled by the agent's fear of failure. RND encourages exploration by providing a curiosity-driven reward based on how difficult it is for a predictor network to mimic a frozen random network's output. This "surprise" bonus enables the agent to navigate complex grids even when the external reward signal is sparse.

3 Methods

Our methodology focuses on the transition from tabular baselines to deep recurrent architectures. The detailed hyperparameters and design choices for each implementation can be found in Appendix A.

3.1 Deep Q-Learning Implementation

The DQN agent minimizes the following temporal difference loss:

$$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right]$$

where θ represents the policy network weights and θ^- represents the frozen target network. Our implementation replaces the standard CNN with tile embeddings suited for symbolic grid observations, allowing for efficient processing of the 5×5 window.

3.2 Architectural Enhancements

Building upon the DQN, we implement three major enhancements:

1. **Recurrency:** We integrate an LSTM layer after the embedding stage to handle the POMDP nature of the vision window.
2. **Wind Awareness:** The current wind direction is passed as a second explicit input through a learned 16-dimensional embedding, allowing the agent to proactively adjust its policy.
3. **Quantile Regression:** This will be explained in the following section.

3.3 QR-DRQN

Standard Q-learning only predicts the *expected* future reward for each action. QR-DRQN improves this by learning a full distribution over returns instead. Concretely, the output head is replaced with one that produces $N = 51$ quantile estimates $\{Z_{\tau_i}(s, a)\}_{i=1}^N$, spaced at $\tau_i = (i - 0.5)/N$, giving a picture of the entire range of possible outcomes for each action.

The network is trained by minimizing the asymmetric Quantile Huber loss: [2]

$$L_{QR}(\theta) = \frac{1}{N^2} \sum_{i=1}^{N} \sum_{j=1}^{N} \rho_{\tau_i}(\delta_{ij})$$

$$\delta_{ij} = r + \gamma Z_{\tau_j}(s', a^*;\, \theta^-) - Z_{\tau_i}(s, a;\, \theta)$$

where $\rho_\tau(u) = |\tau - \mathbf{1}(u < 0)| \cdot \mathcal{L}_\kappa(u)$ and $\mathcal{L}_\kappa$ is the Huber loss with threshold $\kappa = 1$.

At inference, the greedy action is selected by averaging over all quantiles. In practice, this makes the agent more reliable when wind randomly redirects its steps, since it learns to account for that variability rather than being surprised by it.

3.4 RND-DRQN

RND-DRQN augments the standard DRQN with a curiosity-driven intrinsic reward signal to combat stagnation under the -0.1 living penalty. Two auxiliary networks are introduced: a **frozen target network** $f_{\bar{\theta}}$ with randomly initialized, non-trainable weights, and a **predictor network** f_ϕ trained to replicate the target's output. Both share the same architecture: vision and wind embeddings concatenated and passed through a two-layer MLP (64 hidden units, ReLU) to a 32-dimensional feature vector.

At each timestep, the intrinsic reward is the mean-squared prediction error: [1]

$$r_t^{\text{int}} = \left\| f_\phi(o_t, w_t) - f_{\bar{\theta}}(o_t, w_t) \right\|^2$$

The agent is trained on the combined reward signal:

$$r_t = r_t^{\text{ext}} + \alpha\, r_t^{\text{int}}, \quad \alpha = 0.5$$

The total training loss is $\mathcal{L} = \mathcal{L}_{\text{DRQN}} + \mathcal{L}_{\text{RND}}$, where $\mathcal{L}_{\text{RND}}$ updates only f_ϕ, leaving $f_{\bar{\theta}}$ frozen throughout training.

4 Experiments and Results

4.1 Experimental Setup

Experiments were conducted on an 8×8 Enhanced Frozen Lake grid, with 12 holes randomly placed at the start of each episode to prevent overfitting to a fixed layout. The agent's observation at each timestep was restricted to a 5×5 tile window centered on its current position, padded with wall tokens at boundaries.

A dynamic "wind" variable stochastically overrides the agent's chosen action with probability $p = 0.05$, pushing the agent in one of four directions (left, down, right, or up) and the active wind direction shifts every $1,000$ episodes.

To discourage stagnation, we applied the following reward structure:

- **Living Penalty:** -0.1 per step.
- **Discovery Bonus:** $+0.5$ whenever the agent reached a new minimum Manhattan distance to the goal.
- **Terminal Rewards:** $+15$ for the Goal and -2 for a Hole.

4.2 Model Progression and Architectures

We evaluated a sequence of agents, evolving from simple baselines into complex hybrid architectures. Table 2 summarizes this progression and the specific benefits provided by each architectural addition.

Table 2: Evolution of Model Architectures

Model Stage	Key Addition	Primary Benefit / Challenge Addressed
Tabular Q-Learning	Lookup table	Baseline; fails on high-dimensional POMDPs.
Standard DQN	Tile embeddings	Handles larger state spaces; lacks temporal memory.
DRQN	+ LSTM layer	Solves partial observability via state sequences.
QR-DRQN	+ Quantile Reg.	Increases robustness to wind-induced uncertainty.
RND-DRQN	+ Curiosity bonus	Prevents local minima in sparse reward maps.

Baselines: Tabular Q-Learning and DQN
As expected, Tabular Q-Learning achieved a success rate of only 17.0%. The high-dimensional state space created by the vision window and wind combinations exceeded the capacity of a lookup table. Standard DQN improved this to 63.0%, but remained limited by its lack of memory; without recurrency, the agent could not distinguish between identical local observations at different global positions.

The Role of Memory and Wind Awareness: DRQN
The introduction of an LSTM layer lead the success rate to jump to 94.0%. The hidden state allowed the agent to "map" its surroundings as it moved. Furthermore, providing **wind direction as an explicit input** had a significant effect; by concatenating wind embeddings, the agent proactively compensated for environmental shifts rather than reacting to them after the fact.

Advanced Hybrid Models: QR-DRQN and RND-DRQN
Both hybrid models achieved a 97.0% success rate, the highest among all learning agents. QR-DRQN showed greater stability in the presence of stochastic wind, while RND-DRQN reached the goal more efficiently in early training by incentivizing the exploration of novel states.

4.3 Evaluation Methodology

We used a custom evaluation script, `evaluate_models.py`, testing each agent over 100 episodes with **fixed seeds** (starting from 2000) to ensure consistent map generation. We also calculated an **Oracle Baseline** using BFS, which has full map visibility and provides a theoretical upper bound for rewards and path efficiency.

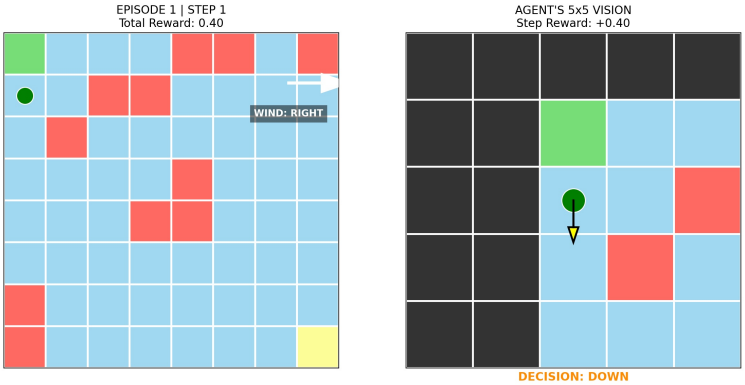

Figure 1: RND-DRQN Evaluation on map8_v5

4.4 Comparison of Results

The results of our evaluation are summarized in Table 3.

Table 3: Performance Comparison of RL Agents on Enhanced Frozen Lake

Model	**Key Architecture**	**Success Rate**	**Avg Reward**
Oracle (BFS)	Shortest Path (Full Map)	100.0%	30.70
Tabular Q-Learning	State-Wind Lookup Table	17.0%	−2.53
Standard DQN	Tile Embeddings + MLP	63.0%	8.65
Standard DRQN	LSTM Memory	94.0%	19.24
QR-DRQN	**Distributional + LSTM**	**97.0%**	**19.81**
RND-DRQN	**Curiosity + LSTM**	**97.0%**	19.76

The results demonstrate a clear progression in performance as architectural complexity increases. The Tabular Q-Learning baseline struggles severely, achieving a 17.0% success rate and negative average rewards, underscoring the inadequacy of lookup tables for partially observable, high-dimensional state spaces. The incorporation of deep neural networks in the Standard DQN raises the success rate to 63.0%; however, the absence of temporal memory restricts its ability to resolve state aliasing. The introduction of recurrent memory via the Standard DRQN yields a profound improvement, elevating the success rate to 94.0% and the average reward to 19.24. Finally, the advanced hybrid models, QR-DRQN and RND-DRQN, further narrow the gap to the optimal performance of the Oracle baseline. By explicitly mitigating environmental stochasticity and promoting exploration in the presence of sparse rewards, both models attain a near-optimal 97.0% success rate and average rewards approaching 20.0, demonstrating their robust capability in navigating challenging POMDP dynamics.

5 Discussion

5.1 LSTM layer in partial observability problem

The substantial improvement from a 63.0% success rate in DQN to 94.0% in DRQN constitutes the most significant performance gain observed in this study. This improvement highlights the "memory threshold" necessitated by partial observability. Within the constrained 5×5 observation window, the agent frequently experiences state aliasing—scenarios where local observations (e.g., exclusively frozen tiles) are visually identical despite corresponding to distinct global coordinates. The standard DQN architecture processes these instances as identical states, precipitating circular trajectories or

repetitive failures. Conversely, the integration of an LSTM layer in DRQN mitigates this issue by maintaining a hidden belief state, which aggregates sequences of observations to effectively infer global coordinates from sequential local movements.

5.2 Risk Management versus Directed Curiosity

The hybrid models, QR-DRQN and RND-DRQN, both converged to a near-optimal 97.0% success rate, albeit through fundamentally divergent mechanisms:

- **QR-DRQN (Risk Awareness):** In environments exhibiting stochastic non-stationary dynamics (i.e., "wind"), transitions are inherently probabilistic. A conventional agent may erroneously attribute a negative value to a generally optimal action if an unpredictable transition forces it into a terminal failure state. QR-DRQN mitigates this by estimating the complete distribution of returns, enabling it to differentiate between actions carrying a marginal probability of catastrophic failure and those with uniformly poor expected returns. This distributional estimation endows the agent with robust resilience against aleatoric uncertainty.
- **RND-DRQN (Curiosity-Driven Exploration):** The implementation of a continuous living penalty frequently induces learned helplessness, wherein agents remain localized near the starting position to preclude the severe penalties associated with terminal failure. Random Network Distillation provides an intrinsic curiosity bonus proportional to the prediction error of a novel state. This intrinsic motivation compels the agent to systematically explore the 8×8 grid, thereby circumventing local minima that typically impede standard architectures.

5.3 Empirical Performance Upper Bound

The convergence of both advanced hybrid models at a 97.0% success rate implies an empirical performance ceiling for the 8×8 environment under the current parameterization. This residual 3.0% margin is likely attributable to overlapping model competencies; once temporal memory establishes a robust 94.0% baseline, the distributional and curiosity-driven enhancements serve primarily to resolve the remaining edge cases. Furthermore, we hypothesize that approximately 2.0% to 3.0% of the procedurally generated maps constitute stochastically impossible configurations, wherein specific combinations of obstacle placement and non-stationary dynamics render the goal mathematically unattainable irrespective of the algorithmic approach employed.

6 Limitations and Future Directions

6.1 Current Limitations

Our BFS Oracle operates under idealized conditions: it has full grid visibility and experiences no wind perturbations during traversal. While this was a deliberate design choice to ensure that the randomly generated map was feasibly, it undermines the fairness of comparison, as pointed out by Dr. Ziping Xu. We acknowledge this as a limitation, and future work should implement a wind-aware BFS or a Monte Carlo Tree Search baseline that operates under identical environmental constraints. Another limitation of our current approach is the computational overhead associated with LSTM layers and Quantile Regression, which increases training latency and memory requirements. Additionally, while our methods proved highly effective on an 8×8 grid, the "risk-averse" nature of QR-DRQN might encounter challenges on much larger maps where the exploration-exploitation trade-off becomes more extreme.

6.2 Future Directions

Future research should focus on scaling these architectures to **larger and more complex maps**, such as 16×16 or 32×32 grids, with a significantly higher density of obstacles and holes. To manage the increased sparsity of rewards in these environments, we propose applying **new reward setting techniques**, such as *Curriculum Learning* or *Hindsight Experience Replay* (HER). By incrementally increasing map complexity or allowing agents to learn from failed episodes, these techniques could bridge the gap to human-level performance on large-scale grids. Furthermore, investigating a "triple

hybrid" model (QR-RND-DRQN) remains a promising path to determine if combining explicit risk-awareness with robust curiosity can finally resolve the remaining 3% failure gap.

7 Conclusion

In this project, we demonstrated that solving complex, stochastic grid-worlds necessitates addressing the "deadly triad" of reinforcement learning: partial observability, non-stationary dynamics, and sparse rewards. By systematically evaluating a progression of agents on the Enhanced Frozen Lake environment, we identified the specific architectural components required to overcome each challenge. Our findings reveal that recurrent memory (LSTM) acts as the critical foundational element, resolving state aliasing and providing a profound 31.0% performance increase over standard feed-forward baselines. Furthermore, achieving near-optimal performance in highly unpredictable settings requires more than just temporal memory. The integration of distributional return estimation (QR-DRQN) provides essential resilience against aleatoric uncertainty, allowing the agent to manage risk rather than merely reacting to it. Concurrently, curiosity-driven exploration (RND-DRQN) actively mitigates the stagnation caused by sparse rewards and harsh living penalties by intrinsically motivating the agent to discover novel states. Ultimately, these advanced hybrid architectures underscore a vital design principle for real-world RL applications: autonomous agents must be equipped not only to maintain an internal representation of their environment, but also to probabilistically evaluate risk and intelligently direct their own exploration.

Code Availability

Our complete codebase, including the enhanced simulation environments, all agent architectures, and evaluation scripts, is publicly available as supplementary material in our GitHub repository: `https://github.com/soph-shen/FrozenLake-RL`.

Acknowledgements

We are grateful to Dr. Ziping Xu for the foundational course content that informed this work and for his guidance throughout the research process. We also thank TA Siwei Han for her helpful feedback and support.

References

[1] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by Random Network Distillation, October 2018.

[2] Will Dabney, Georg Ostrovski, David Silver, and Rémi Munos. Implicit Quantile Networks for Distributional Reinforcement Learning, June 2018. Comment: ICML 2018.

[3] Matthew Hausknecht and Peter Stone. Deep Recurrent Q-Learning for Partially Observable MDPs, January 2017.

[4] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. Standard DQN: Introduces experience replay and target networks to stabilize Q-learning.

[5] Sindhu Padakandla. A Survey of Reinforcement Learning Algorithms for Dynamically Varying Environments. *ACM Computing Surveys*, 54(6):1–25, July 2022.

A Implementation Details

Table 4: Q-Learning Implementation: Design Choices

Setting	**Value**	**Reason**
State key s	$(t_1, \ldots, t_{25}, w)$	Flattened 5×5 tile observations plus wind direction w.
Learning rate α	0.1	Balances convergence stability and learning speed.
Discount factor γ	0.99	Ensures long-term rewards (reaching the goal) are prioritized.
Exploration ε	$1.0 \rightarrow 0.05$	Exponential decay with a 5% floor to maintain minimal exploration.
Max episodes	150,000	Consistent training duration for all baseline comparisons.

Table 5: DQN Implementation: Design Choices

Setting	**Value**	**Reason**
Tile Embeddings	5×16	Replaces CNN with learned vectors for grid symbols.
Wind Embeddings	4×16	Explicit feature for non-stationary wind direction.
Architecture	$416 \rightarrow 128 \rightarrow 128 \rightarrow 4$	MLP with two hidden layers and ReLU activations.
Replay Buffer	100,000	Sufficient capacity to break temporal correlations.
Batch Size	64	Standard mini-batch size for stable gradient updates.
Learning Rate α	1×10^{-4}	Optimal for Adam optimizer stability in this domain.